

Reputation & Feedback Intelligence Engine

How a completed visit becomes scored feedback, routed four ways, with a draft reply waiting and a manager alerted when it matters.

Date 22 September 2026 **Platform** n8n **Delivery** Group work **Source** Capstone Project Information.txt

Context

A multi-location car care and auto repair chain has two problems. Follow-up after a visit is inconsistent, so a bad review appears online before anyone internally has heard a word from that customer. And all feedback is handled identically — a mild grumble about a delay gets the same treatment as a furious complaint about a botched job.

This system closes that loop. Every completed visit triggers a feedback request. Every reply is read and scored on **two axes** rather than sorted into two buckets. What happens next depends on the score **and** on whether this customer has complained before. Everything lands on one dashboard, filterable by branch.

The standing rule. Nothing is ever published publicly by the system, and nothing is ever sent to a customer by the system. A human does both. The system's job is to notice, score, route, and prepare — never to act on the customer's behalf.

What is settled, and what is not

DECISION	CHOICE
Platform	n8n
Audience	Course capstone — graded on a working end-to-end demo, not production hardening
Delivery	Group work, more than one person building
Hosting	n8n Cloud to build and demo, then export to self-hosted Docker on Oracle Cloud
Domain	Already owned — used for the form, the dashboard and n8n itself once migrated
Model	Claude (Anthropic) — <i>assumed, to confirm</i>

Four questions for the group

QUESTION	OPTIONS
Datastore + dashboard	Airtable + Interface Designer · Supabase + custom dashboard · Supabase + hosted page · Sheets + Looker Studio
Request and reply channel	n8n Form link (<i>recommended</i>) · email out and reply in · Telegram bot · simulated webhook
Manager alert channel	Telegram · email · Slack · both
"Ready to post" content	The customer's raw words, or a lightly polished review drafted from them

Three things in the brief that are easy to miss

There are four routes, not three

The brief's flow section lists three — positive, negative-mild, negative-severe. But point 8 adds a fourth: anything the system cannot confidently score is **flagged for a human rather than guessed at**. It is built here as a first-class route with its own queue, not as an error handler bolted on the side.

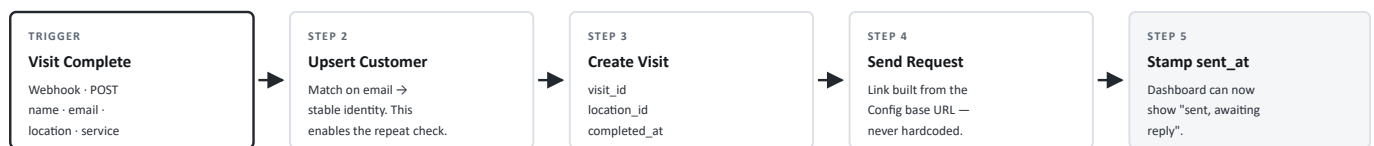
Scoring is two numbers, not one

"Score how positive or negative it is, *and* how severe." Those are different axes. "Fine, a bit slow" is negative but low severity; "you ruined my car" is negative and high severity. Collapsing them into a single value would be exactly the flattening the brief objects to.

The repeat-customer rule is an override, not an input

Point 4 says escalate "regardless of how mild this particular message reads". So history does not nudge the score — it **bypasses the severity threshold entirely**. This distinction is what makes the third demo case visibly different from the second rather than coincidentally different.

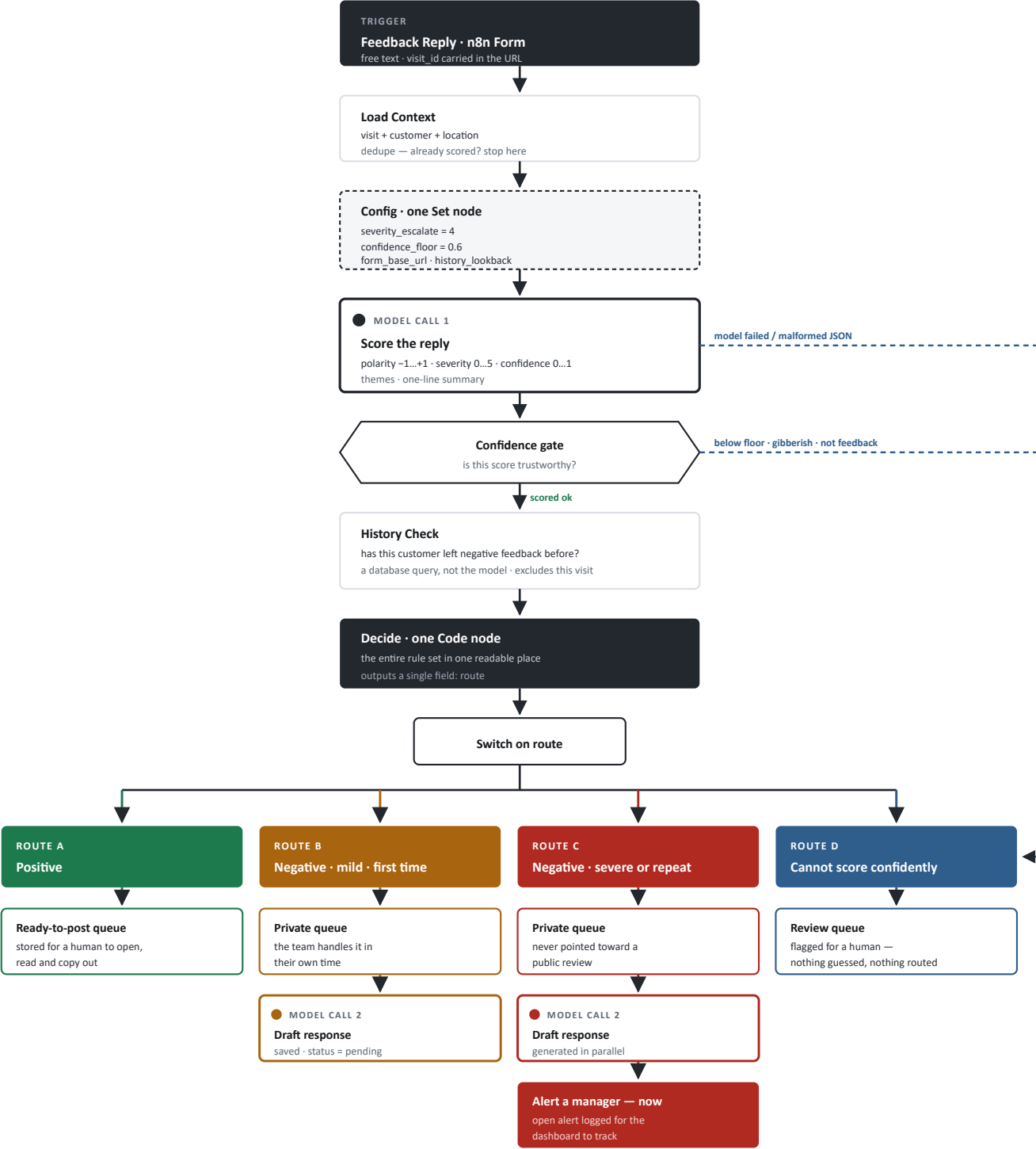
Workflow 1 — Visit complete becomes a feedback request



The webhook stands in for the shop's job management system. Deliberately simple — the brief asks only for a reliable trigger, and the interesting work is all downstream.

Why the customer record comes before the visit record. The repeat-customer check later in the flow is only possible if the same person is recognisable across two separate visits. That identity has to be established here, on the way in, matched on a stable key. Get this wrong and route C never fires, no matter how correct the routing logic is.

Workflow 2 — Reply becomes a scored, routed decision



Route A is never published automatically. Routes B and C are never sent automatically. Both wait for a human.

Solid lines are the scored path. Dashed blue is the bypass: a failed or untrustworthy score skips the routing rules entirely and goes straight to a human.

The decision logic

This is the single Code node, written out. **Order matters — first match wins.** Putting the whole rule set in one place is deliberate: it is the part a reviewer will ask about, and it should be possible to point at one node and read it top to bottom.

#	CONDITION	ROUTE	WHY IT SITS IN THIS POSITION
1	Model failed, or confidence < 0.6 , or not really feedback	D — review	Checked first, so a broken score can never reach the routing rules below
2	polarity < 0 and customer has a prior negative	C — escalate	Above the severity test deliberately. This is the override — a mild second complaint still escalates
3	polarity < 0 and severity ≥ 4	C — escalate	Severity on its own is enough, no history required
4	polarity < 0	B — private queue	First time, lower severity. Queued for the team; a draft is still generated
5	polarity > 0	A — ready to post	Stored for a human to read and copy out
6	Anything left — neutral	D — review	Neutral is not positive, and route A feeds something a human may publish

Two points worth challenging before we build

Rule 2 sits above rule 3. This is the brief's point 4 taken literally. Flip the order and a mild repeat complaint gets severity-tested first, falls through to the ordinary queue, and the third demo case stops looking different from the second. This ordering is the thing that makes the repeat path visibly its own path.

Rule 6 sends neutral to review rather than to the positive queue. "The car's ready then?" is not a glowing review, and route A feeds something a human may publish. Deliberately cautious — worth confirming that everyone agrees.

Four design positions

The routing decision lives in one Code node, not a tree of IF nodes

The rule is four-way with an override. Expressed as nested IF nodes that is four levels of branching smeared across the canvas — hard to read, hard to change, and hard to point at when someone asks where the logic is. One Code node emitting a route field, then a single Switch on that field, puts the whole decision in one place. For a graded project that legibility is worth a great deal.

A Basic LLM Chain with a structured output parser, not an AI Agent

This is one classification call, not a multi-step tool-using task. An Agent adds latency, cost and nondeterminism — and nondeterminism is the last thing wanted in a demo that must behave identically three times in front of an audience.

Two model calls, not one

Score first, draft second, and only on negatives. Nothing is spent drafting replies to happy customers; the scoring call stays tightly constrained while the drafting call is free to be warm and specific; and if drafting fails there is still a correct score and a correct route. One call doing both jobs couples two failures together.

Every threshold lives in one Config node

Severity cut-off, confidence floor, history lookback, the form's base URL — one Set node, none of them buried inside expressions. A reviewer can see every rule at a glance, and the migration to Oracle becomes a one-value change rather

than a hunt.

Building now for the move to Oracle Cloud

Building on n8n Cloud and later self-hosting on Docker is the right sequence — public HTTPS and zero setup while the deadline is live, then something owned outright that costs nothing. But **four things do not survive an export**, and each is cheap to design around now and painful to retrofit later.

WHAT BREAKS	WHAT TO DO ABOUT IT NOW
Credentials do not export	Workflow JSON is exported with credential references stripped. Every credential — Anthropic, the datastore, Gmail, Telegram — is re-created by hand on the Oracle instance. Keep a written list as each one is added; reverse-engineering it later from red node badges is miserable.
Webhook and form URLs change	Cloud gives <code><name>.app.n8n.cloud</code> ; Oracle gives your own domain. Anything hardcoding the cloud URL breaks silently on moving day. Assemble the form link from a single base-URL value in the Config node. Because the domain is already owned, that value can point at its permanent home from the start and never change at all.
Data held inside n8n	With Airtable or Supabase the move touches nothing — the workflows point at the same external base either way. Using n8n's own Data Tables would mean migrating data by hand as well as workflows. An argument for the external datastore already being considered.
Node version parity	Export from a newer n8n into an older self-hosted build and <code>typeVersion</code> mismatches show up as parameters that quietly do not apply. Note the cloud version before exporting and pin the Docker image to match, rather than taking <code>latest</code> .

One more, unrelated to the export: Oracle's Always Free instances are arm64. The official n8n image is multi-arch so that is fine, but any sidecar container added alongside it needs an arm64 image too.

What the owned domain unlocks

A domain plus a free Oracle box means hosting web pages at no cost, which removes the main practical objection to *building* a dashboard rather than configuring one. It also means a customer-facing form on your own domain rather than an `app.n8n.cloud` link — a small thing that reads noticeably better in a demo.

SUBDOMAIN	SERVES
<code>n8n.<domain></code>	The n8n instance, once migrated
<code>feedback.<domain></code>	The customer-facing feedback form
<code>dashboard.<domain></code>	The management dashboard

The trade-off is unchanged, though: a custom dashboard is still a second build. The domain removes its hosting cost and its deployment question, not its effort.

Cost

This can be built for roughly five dollars. Cost should not decide any of the four open questions — the two gotchas below matter considerably more than the money does.

Running the model

Claude Opus 5 at \$5 per million input tokens and \$25 per million output. Token counts are estimates based on a scoring prompt carrying the rubric plus the customer's reply.

CALL	INPUT	OUTPUT	PER CALL
1 — Score the reply	~600 tok	~150 tok	~\$0.007

2 — Draft response (<i>negatives only</i>)	~700 tok	~250 tok	~\$0.010
Positive item — one call	—	—	~\$0.007
Negative item — both calls	—	—	~\$0.017

Round it to **two cents per piece of feedback**. Three hundred runs across the whole build — development, testing, re-runs, the live demo — comes to about **\$5**. Anthropic's minimum credit purchase is \$5, so that figure is both the floor and, realistically, the ceiling.

Everything else

COMPONENT	COST ON THIS PATH	NOTES
n8n Cloud — build phase	Free 14 days, then €24/mo	Trial gives Pro features, 1,000 executions, no credit card
n8n on Oracle Cloud — after	£0	Always Free ARM instance; Community Edition has unlimited executions
Airtable	Free	1,000 records per base, one interface , 5 editors
Supabase	Free	500 MB, 2 active projects
Gmail / Telegram	Free	Sending limits far above anything needed
Domain + dashboard hosting	£0	Already owned; served from the same Oracle box

The real constraint is a deadline, not a bill

The n8n Cloud trial is 14 days. Three possible outcomes, and the group should pick one on day one rather than discover it on day thirteen: finish and demo inside 14 days and pay nothing; overrun and pay a single month of Starter at €24, which is trivial; or migrate to Oracle earlier than planned and absorb that setup work mid-project.

Two gotchas that matter more than the price

Supabase free projects pause after a week of inactivity. Build in October, demo in November, and the database is asleep. It does unpause — but discovering that five minutes before presenting is a bad afternoon. If Supabase is chosen, either someone touches the project weekly or the demo runbook opens with a wake-up step.

Airtable's free plan allows exactly one interface. An interface holds many pages, so all six dashboard requirements still fit inside it — but there is no room for a second interface alongside, for a different audience or as a spare.

The demo, and why it needs a seeder

The brief is explicit: show a positive case, a first-time negative case, and a repeat-negative case, so all three paths can be seen behaving differently rather than described as doing so.

There is an ordering constraint hiding in that requirement. The repeat case only behaves as a repeat if that customer already has a negative on file. So the demo cannot simply fire three messages — the repeat customer's *first* complaint has to have been processed before their second arrives. Clicking through that live is fragile and slow.

Hence a fourth workflow: a **demo seeder** on a manual trigger that fires every canonical case in the correct order and leaves the dashboard showing all four routes populated. It makes the demo a single click, repeatable, and identical every time it runs.

The four workflows

#	WORKFLOW	PURPOSE
1	Visit Complete → Request	Webhook standing in for the shop's job management system
2	Reply → Score → Route	The core pipeline — scoring, history, routing, drafting, alerting
3	Manager actions	Mark an alert acted on, a draft sent, a review posted (<i>may live in the dashboard instead</i>)
4	Demo seeder	One click, every case, correct order

What is not yet written

The data model, the node-by-node build order, the dashboard specification and the verification steps all follow once the four open questions on page 2 are settled. Those four are the blockers; everything else in this document is stable enough to build against.

Prices checked 22 September 2026 against n8n, Airtable and Supabase published pricing, and Anthropic's published API rates. Figures for model usage are estimates based on assumed prompt sizes, not measured runs.